

DEFENSE LOGISTICS & ASSET MANAGEMENT SYSTEM

Technical Architecture, Data Models, RBAC Security & Deployment Guide

EXECUTIVE SUMMARY

A comprehensive enterprise-grade logistics and armory tracking solution built for defense assets, motor pools, personnel custody, and maintenance workflows. This report provides complete technical documentation regarding system architecture, data models, role-based access enforcement, audit logging, setup instructions, REST API interfaces, and verified operational credentials.

System Designation:

DLAMS Command HQ v1.0.0

Technology Stack:

React 18, Node.js / Express, MySQL & SQLite

Target Environment:

College Capstone / Enterprise Demonstration

Security Classification:

Restricted Educational / Fictional Logistics Data

1. Project Overview

1.1 System Description

The **Defense Logistics & Asset Management System (DLAMS)** is an end-to-end command portal designed for military armories, supply depots, and forward operating bases (FOBs). It solves the operational challenge of tracking, allocating, maintaining, and transferring critical equipment, vehicles, armament stockpiles, and personnel assignments under an auditable digital ledger.

The system features **16 fully interactive pages** including live telemetry dashboards, asset intake forms, vehicle registries, sensitive communications gear rosters, maintenance work order pipelines, custody transfer workflows, and system audit logs.

1.2 Core Capabilities

- **Dual-Phase Authentication Gateway:** Features both an *Officer Registration / Enlistment Process* and a *Standard Operational Cipher Sign-In* with automated Service ID generation.
- **Complete Asset Lifecycle Tracking:** Manages lifecycle phases from Acquisition & Commissioning $\rightarrow$ Personnel Assignment $\rightarrow$ Corrective/Routine Maintenance $\rightarrow$ Inter-Base Relocation $\rightarrow$ Decommissioning.
- **Tactical Visual Center:** Real-time SVG charts for Readiness distribution, Category volume, Maintenance statuses, and 6-month logistics activity trends.
- **Automated Persistence Fallback:** Automatically connects to MySQL if present, or transparently falls back to an embedded SQLite database engine for instant zero-configuration demonstration.

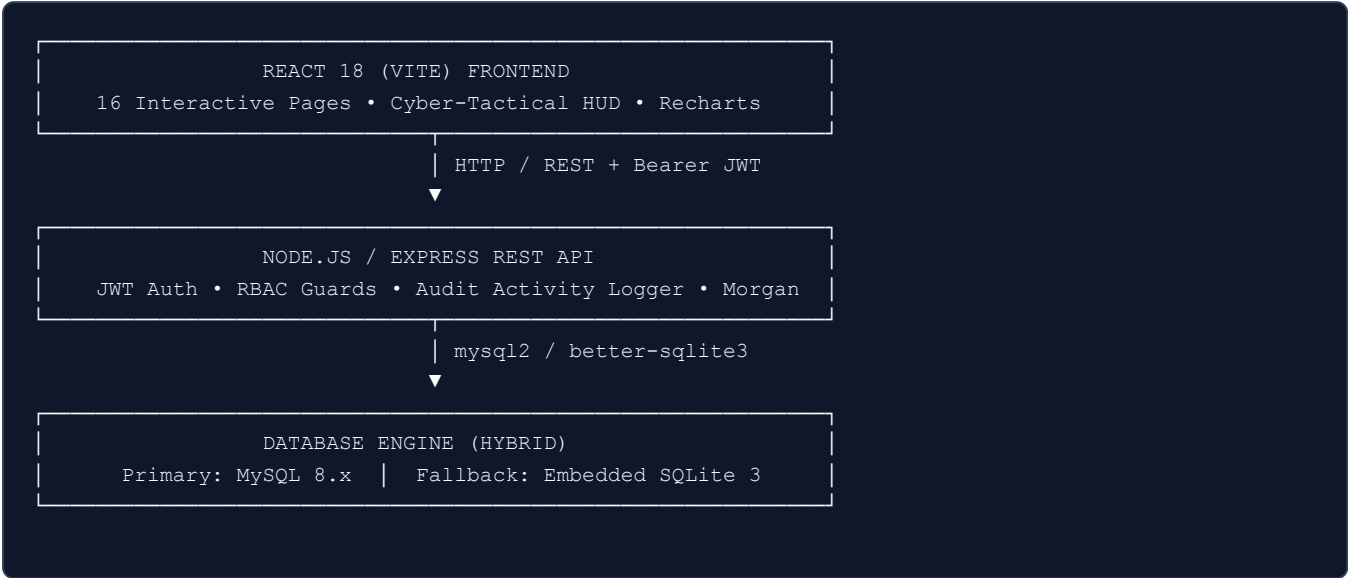
1.3 Assumptions & Constraints

- **Synthetic Educational Data:** All equipment serial numbers, military unit designations, personnel rosters, and dollar valuations are strictly fictional and designed for evaluation.
- **Authentication Model:** Stateless JSON Web Tokens (JWT) with 10-round bcrypt password hashing. Physical Common Access Card (CAC) / PIV authentication is modeled logically via unique Service IDs.
- **Export Mechanisms:** Manifest exports generate compliant CSV documents synchronously in memory.

Compliance Note: This application contains no real military operational information, weapons deployment coordinates, or classified national defense material.

2. Tech Stack & Architecture

2.1 High-Level Architecture Diagram



2.2 Technology Selection & Rationale

Tier	Technology	Technical Justification
Frontend	React 18 + Vite	Sub-second HMR development builds, component reusability, deterministic state synchronization, and fast client-side routing.
Styling	Vanilla Tactical CSS	Custom-engineered military HUD design system with glow filters, tactical badges, and responsive drawers without bulky UI dependencies.
Visualizations	Recharts (SVG)	High-performance, declarative charts for asset operational readiness, depot capacity, and monthly transfer trends.
Backend API	Node.js + Express	Asynchronous event-loop architecture optimal for low-latency JSON REST endpoints and middleware processing.
Security	JWT + bcryptjs	Stateless tokens with cryptographically signed payload claims for role clearance, combined with salted password encryption.
Database	Dual Engine: MySQL + SQLite	MySQL provides production-grade relational constraints and foreign keys. SQLite (via better-sqlite3) guarantees instant zero-setup demonstration capability.

3. Data Models & Database Schema

The database consists of **10 relational entities** with enforced foreign key constraints, cascading updates, and indexing:

Core Schema Entities:

Table	Key Columns	Relationships & Constraints
users	id, username, email, password, role, rank_title, military_unit, service_id, status	Unique on username , email , and service_id . Role enum: Admin , Manager , Staff .
personnel	id, service_number, full_name, rank_title, unit, role_assignment, clearance_level, status	Linked as asset custodians, drivers, and transfer recipients.
assets	id, asset_code, name, type, serial_number, quantity, status, location, assigned_to_id, value_usd	Master inventory record. Foreign key: assigned_to_id -> personnel(id) .
vehicles	id, asset_id, vehicle_type, registration_no, engine_no, mileage_km, assigned_driver_id	Specialized entity. Foreign keys: asset_id -> assets(id) , assigned_driver_id -> personnel(id) .
equipment	id, asset_id, category, calibration_date, battery_health, serial_no, storage_locker	Specialized equipment telemetry. Foreign key: asset_id -> assets(id) .
maintenance	id, work_order_no, asset_id, maintenance_type, technician_name, cost, status	Tracks corrective repair and routine inspection cycles. Foreign key: asset_id -> assets(id) .
transfers	id, transfer_code, asset_id, source_location, destination_location, requested_by_id, approved_by_id	Custody transfer lifecycle. Foreign keys to assets , users (requester/approver), and personnel .
activity_logs	id, user_id, user_name, action, module, details, ip_address, created_at	System audit trail. Foreign key: user_id -> users(id) (ON DELETE SET NULL).
notifications	id, user_id, title, message, type, is_read, link	System alerts. Foreign key: user_id -> users(id) .

4. Role-Based Access Control (RBAC)

Clearance levels are strictly governed by an RBAC matrix enforced both server-side via Express middleware and client-side via React Route Guards.

4.1 Clearance Permissions Matrix

Operation / Resource	Staff (Level 1)	Manager (Level 2)	Admin (Level 3)
View Dashboard KPIs & Telemetry	✓	✓	✓
View Inventory, Vehicles & Armory	✓	✓	✓
Log Maintenance Work Orders	✓	✓	✓
Create Asset Intake & Edit Specs	✗ (Read Only)	✓	✓
Approve / Reject Logistics Transfers	✗	✓	✓
Export Classified Manifests (CSV)	✗	✓	✓
Permanent Asset Decommissioning (Delete)	✗	✗	✓ (Admin Only)
User Account Provisioning & Role Edits	✗	✗	✓ (Admin Only)
System Security Audit Logs Inspection	✗	✗	✓ (Admin Only)

4.2 Middleware Enforcement Pattern

```
// backend/middleware/auth.js
function authorize(allowedRoles = []) {
  return (req, res, next) => {
    if (!req.user || !allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `Security Clearance [${req.user?.role || 'NONE'}] insufficient.`
      });
    }
    next();
  };
}
```

5. API & Transaction Audit Logging

All mutating operations (writes, updates, deletions, and clearance validations) trigger automated logging into the `activity_logs` ledger via `logActivity()` in `backend/middleware/errorHandler.js` :

```
async function logActivity(userId, userName, action, moduleName, details, ip) {  
  await db.query(  
    `INSERT INTO activity_logs (user_id, user_name, action, module, details, ip_address)  
    VALUES (?, ?, ?, ?, ?, ?)` ,  
    [userId, userName, action, moduleName, details, ip || '127.0.0.1']  
  );  
}
```

Tracked Event Categories:

- **Authentication:** LOGIN_SUCCESS , FAILED_LOGIN_ATTEMPT , REGISTER_SUCCESS , LOGOUT .
- **Assets:** CREATE_ASSET , UPDATE_ASSET , DELETE_ASSET .
- **Transfers:** REQUEST_TRANSFER , APPROVE_TRANSFER , REJECT_TRANSFER .
- **Maintenance:** CREATE_WORK_ORDER , COMPLETE_WORK_ORDER .
- **Administration:** CREATE_USER , UPDATE_USER , DELETE_USER .

6. Setup & Execution Instructions

6.1 Prerequisites

- **Node.js:** v18.0.0 or higher
- **npm:** v9.0.0 or higher
- **Browser:** Chrome, Firefox, Edge, or Safari
- **Database (Optional):** MySQL 8.0+ running on port 3306 (if MySQL is not active, the system automatically runs on embedded SQLite).

6.2 One-Command Concurrent Launch (Recommended)

From the root project directory:

```
# Step 1: Install all dependencies across root, backend, and frontend
npm run install:all

# Step 2: Start both Backend (port 5001) and Frontend (port 5173) concurrently
npm run dev
```

6.3 Manual Step-by-Step Launch

1. Backend Setup:

```
cd backend
npm install
npm start
```

Backend runs on `http://localhost:5001`. On initial startup, the database is automatically verified and seeded.

2. Frontend Setup:

```
cd frontend
npm install
npm run dev
```

Frontend runs on `http://localhost:5173`.

7. Documented Key API Endpoints

Method	Endpoint	Clearance	Description
POST	/api/auth/register	PUBLIC	Enlist new officer and generate Service ID.
POST	/api/auth/login	PUBLIC	Authenticate using username or email + cipher.
GET	/api/dashboard/stats	ALL ROLES	Retrieve KPI summaries and chart statistics.
GET	/api/assets	ALL ROLES	Paginated asset inventory with filters.
POST	/api/assets	MANAGER+	Commission new asset into command inventory.
DELETE	/api/assets/:id	ADMIN ONLY	Permanently decommission asset record.
POST	/api/transfers	ALL ROLES	Request equipment custody transfer.
PATCH	/api/transfers/:id/status	MANAGER+	Approve, reject, or complete transfer.
GET	/api/reports/export/csv	MANAGER+	Export live inventory ledger to CSV file.
GET	/api/activity-logs	ADMIN ONLY	Retrieve complete audit trail of system events.

8. Verified Working Login Credentials

The platform includes 3 pre-seeded clearance accounts ready for immediate testing. In addition, the login interface features **1-click autofill buttons**:

Role	Rank & Call-Sign	Email Address	Password Cipher	Operational Clearance
ADMIN	Colonel Mitchell col_mitchell	admin@example.com	Admin@123	Full Command HQ authority, asset deletion, user provisioning, system audits.
MANAGER	Major Vance maj_vance	manager@example.com	Manager@123	Depot command, transfer approvals, equipment editing, manifest exports.
STAFF	Staff Sgt. Miller sgt_miller	staff@example.com	Staff@123	Armory intake, transfer requests, maintenance work order creation.

Pro Tip: You can also click the **"Register Process"** tab on the login screen to register your own custom call-sign, rank, military unit, and password cipher.